Python Practice Exercises for IoT Career

Week 1: Hands-On Coding (No Hardware Needed Yet)

WHY YOU NEED PRACTICE (Not Just Videos)

What happens when you only watch:

- ❌ Brain: "Yeah, I understand this"
- ❌ Reality: Can't write code when you need to
- ❌ Interview: "Write a function to parse sensor data" → You freeze

What happens when you practice:

- ✅ Brain: Muscle memory for syntax
- ✅ Reality: Can modify code confidently
- ✅ Interview: "Parse sensor data" → You write it in 5 minutes

The rule: For every 1 hour of video, spend 2 hours coding

PRACTICE METHOD: 3-Step Loop

For each exercise below:

Step 1: Try it yourself (30 min)

- Don't Google immediately
- Struggle for at least 15-20 minutes
- Make mistakes, get errors

Step 2: Get it working (20 min)

- Now Google specific errors
- Check documentation
- Compare with example solutions

Step 3: Extend it (10 min)

- Modify the exercise
- Add one new feature
- Break it and fix it again

Total per exercise: ~1 hour

EXERCISE 1: Temperature Data Simulator

IoT Relevance: Simulating sensor readings before you have real hardware

Task:

Create a Python script that generates fake temperature readings like a sensor would.

Requirements:

```
python

# Your script should:
# 1. Generate random temperature between 20-35°C
# 2. Create 10 readings
# 3. Each reading has timestamp
# 4. Print in this format:
#    2026-01-24 10:30:15 - Temperature: 24.5°C

# Hints:
# - Use random module: import random
# - Use datetime module: from datetime import datetime
# - random.uniform(20, 35) gives decimal number
# - datetime.now() gives current time
```

Starter Code:

```
python

import random
from datetime import datetime

# TODO: Create a function that generates one temperature reading
def generate_temperature():
    # Your code here
    pass

# TODO: Generate 10 readings
for i in range(10):
    # Your code here
    pass
```

Expected Output:

```
2026-01-24 10:30:15 - Temperature: 24.5°C
2026-01-24 10:30:15 - Temperature: 28.3°C
2026-01-24 10:30:15 - Temperature: 22.1°C
...
```

Extension Challenge:

- Add humidity (40-80%)
- Save readings to a text file
- Calculate average temperature

EXERCISE 2: JSON Sensor Data Handler

IoT Relevance: ALL IoT data is sent as JSON. This is critical.

Task:

Work with sensor data in JSON format (how AWS IoT Core receives data)

Requirements:

```python
# Given this sensor reading:
sensor_data = {
    "device_id": "ESP32_001",
    "location": "Zone_A",
    "timestamp": "2026-01-24T10:30:15",
    "temperature": 24.5,
    "humidity": 65.2,
    "status": "online"
}

# Your tasks:
# 1. Print just the temperature
# 2. Print device_id and location
# 3. Check if temperature > 30, print "HIGH TEMP ALERT"
# 4. Add a new field "battery": 85
# 5. Convert to JSON string and print
```

Starter Code:

```python
import json

sensor_data = {
    "device_id": "ESP32_001",
    "location": "Zone_A",
    "timestamp": "2026-01-24T10:30:15",
    "temperature": 24.5,
    "humidity": 65.2,
    "status": "online"
}

# Task 1: Print temperature
# Your code here

# Task 2: Print device and location
# Your code here

# Task 3: Temperature check
# Your code here

# Task 4: Add battery field
# Your code here

# Task 5: Convert to JSON string
json_string = json.dumps(sensor_data)
print(json_string)
```

Extension Challenge:

- Create a list of 5 sensor readings

- Loop through and print only readings with temp > 25
- Save all readings to a JSON file

EXERCISE 3: Multiple Sensor Data Parser

IoT Relevance: Processing data from multiple devices simultaneously

Task:

Parse data from multiple sensors and find issues

Data:

```python
sensor_readings = [
    {"device": "ESP32_001", "temp": 24.5, "humidity": 65, "location": "Office"},
    {"device": "ESP32_002", "temp": 35.2, "humidity": 45, "location": "Server Room"},
    {"device": "ESP32_003", "temp": 22.1, "humidity": 70, "location": "Hallway"},
    {"device": "ESP32_004", "temp": 28.5, "humidity": 55, "location": "Conference"},
    {"device": "ESP32_005", "temp": 31.5, "humidity": 40, "location": "Lab"}
]
```

Requirements:

```python
# 1. Find all devices with temperature > 30°C
# 2. Calculate average temperature across all devices
# 3. Find the hottest location
# 4. Count how many devices are in normal range (20-28°C)
# 5. Create alert message for any device > 30°C
```

Starter Code:

```python
```

```
sensor_readings = [
    # ... data from above
]

# Task 1: Find hot devices
hot_devices = []
for reading in sensor_readings:
    # Your code here
    pass

print("Hot devices:", hot_devices)

# Task 2: Average temperature
# Your code here

# Task 3: Hottest location
# Your code here

# Task 4: Normal range count
# Your code here

# Task 5: Alert messages
# Your code here
```

Expected Output:

```
Hot devices: ['ESP32_002', 'ESP32_005']
Average temperature: 28.36°C
Hottest location: Server Room (35.2°C)
Normal devices: 3
ALERT: ESP32_002 in Server Room: 35.2°C - COOLING REQUIRED
ALERT: ESP32_005 in Lab: 31.5°C - CHECK HVAC
```

Extension Challenge:

- Add humidity alerts (< 30% or > 70%)
- Sort devices by temperature (hottest first)
- Export alerts to a text file

EXERCISE 4: Reading Sensor Data from File

IoT Relevance: Processing historical sensor logs

Task:

Read sensor data from a CSV file and analyze it

Setup:

First, create a file called `sensor_log.csv`:

```
csv
```

```
timestamp,device,temperature,humidity
2026-01-24 08:00:00,ESP32_001,22.5,60
2026-01-24 08:15:00,ESP32_001,23.1,62
2026-01-24 08:30:00,ESP32_001,24.8,58
2026-01-24 08:45:00,ESP32_001,26.2,55
2026-01-24 09:00:00,ESP32_001,28.5,52
2026-01-24 09:15:00,ESP32_001,30.1,50
2026-01-24 09:30:00,ESP32_001,31.5,48
```

Requirements:

```
python

# 1. Read the CSV file
# 2. Find maximum temperature recorded
# 3. Find minimum humidity
# 4. Calculate temperature increase rate (°C per hour)
# 5. Identify when temperature crossed 25°C threshold
```

Starter Code:

```python
# Read file line by line
with open('sensor_log.csv', 'r') as file:
    lines = file.readlines()

# Skip header (first line)
data_lines = lines[1:]

temperatures = []
timestamps = []

for line in data_lines:
    # Split by comma
    parts = line.strip().split(',')
    timestamp = parts[0]
    temp = float(parts[2])

    temperatures.append(temp)
    timestamps.append(timestamp)

# Task 1: Max temperature
max_temp = max(temperatures)
print(f"Maximum temperature: {max_temp}°C")

# Task 2: You continue...
# Your code here
```

Extension Challenge:

- Calculate average temp for each hour
- Detect sudden temperature jumps (>2°C in 15 min)
- Write analysis results to new file

EXERCISE 5: MQTT Message Simulator

IoT Relevance: Understanding MQTT message structure before using real broker

Task:

Simulate publishing sensor data to MQTT topics (without actual MQTT broker)

Requirements:

```
python

# 1. Create function to format sensor reading as MQTT message
# 2. Generate topic name: "building/zone_A/temperature"
# 3. Create message payload in JSON
# 4. Simulate publishing 5 messages with different values
# 5. Print topic and payload for each
```

Starter Code:

```python
import json
import random
from datetime import datetime

def create_mqtt_message(zone, sensor_type, value):
    """
    Create MQTT topic and payload

    Args:
        zone: Location (e.g., "zone_A")
        sensor_type: Type of sensor (e.g., "temperature")
        value: Sensor reading

    Returns:
        topic (str), payload (dict)
    """
    # TODO: Create topic string
    topic = f"building/{zone}/{sensor_type}"

    # TODO: Create payload dictionary
    payload = {
        "timestamp": datetime.now().isoformat(),
        "value": value,
        "unit": "celsius" if sensor_type == "temperature" else "percent"
    }

    return topic, payload

# Simulate publishing 5 temperature readings
for i in range(5):
    temp = random.uniform(20, 30)
    topic, payload = create_mqtt_message("zone_A", "temperature", temp)

    print(f"Publishing to topic: {topic}")
    print(f"Payload: {json.dumps(payload, indent=2)}")
    print("-" * 50)
```

Extension Challenge:

- Add multiple zones (zone_A, zone_B, zone_C)
- Add different sensor types (temperature, humidity, co2)
- Create function to parse topic and extract zone name
- Simulate subscribing (filtering messages by topic pattern)

EXERCISE 6: Energy Consumption Calculator

IoT Relevance: Processing sensor data to calculate business value ($$)

Task:

Calculate energy cost from HVAC power sensor readings

Requirements:

```
python

# Given:
# - Current sensor reading in Amperes
# - Voltage (always 220V in your case)
# - Power factor (0.85 for HVAC)
# - Electricity rate: PKR 20 per kWh
# - Runtime in hours

# Calculate:
# 1. Power consumption in Watts
# 2. Energy consumption in kWh
# 3. Cost in PKR
# 4. Compare with baseline and show savings %
```

Starter Code:

```
python
```

```python
def calculate_energy_cost(current_amps, voltage, power_factor, runtime_hours, rate_per_kwh):
    """
    Calculate energy cost from sensor readings

    Args:
        current_amps: Current reading from sensor (A)
        voltage: Voltage (V)
        power_factor: Power factor (0-1)
        runtime_hours: Hours of operation
        rate_per_kwh: Cost per kWh

    Returns:
        dict with power, energy, cost
    """
    # Power (W) = Voltage × Current × Power Factor
    power_watts = voltage * current_amps * power_factor

    # Energy (kWh) = Power (kW) × Time (hours)
    power_kw = power_watts / 1000
    energy_kwh = power_kw * runtime_hours

    # Cost = Energy × Rate
    cost = energy_kwh * rate_per_kwh

    return {
        "power_watts": round(power_watts, 2),
        "energy_kwh": round(energy_kwh, 2),
        "cost": round(cost, 2)
    }

# Example: AHU running at 15A for 10 hours
result = calculate_energy_cost(
    current_amps=15,
    voltage=220,
    power_factor=0.85,
    runtime_hours=10,
    rate_per_kwh=20
)

print(f"Power: {result['power_watts']} W")
print(f"Energy: {result['energy_kwh']} kWh")
print(f"Cost: PKR {result['cost']}")

# TODO: Now process multiple readings and calculate total daily cost
daily_readings = [
    {"hour": "08:00", "current": 15.2},
    {"hour": "09:00", "current": 16.1},
    {"hour": "10:00", "current": 17.5},
    {"hour": "11:00", "current": 18.2},
    {"hour": "12:00", "current": 19.1},
    {"hour": "13:00", "current": 18.8},
    {"hour": "14:00", "current": 17.2},
    {"hour": "15:00", "current": 16.5},
]

# Your task: Calculate total daily cost
# Your code here
```

Extension Challenge:

- Compare with baseline (e.g., last year same day)
- Calculate savings percentage
- Identify peak consumption hours
- Generate cost optimization recommendations

EXERCISE 7: Threshold Alert System

IoT Relevance: Real-time monitoring and alerts (foundation for AWS Lambda logic)

Task:

Create a system that monitors sensor readings and generates alerts

Requirements:

```
python

# Rules:
# - Temperature > 30°C → "HIGH TEMPERATURE ALERT"
# - Temperature < 18°C → "LOW TEMPERATURE ALERT"
# - Humidity > 70% → "HIGH HUMIDITY ALERT"
# - Humidity < 30% → "LOW HUMIDITY ALERT"
# - If both temp and humidity are out of range → "CRITICAL ALERT"

# Create function that:
# 1. Takes sensor reading
# 2. Checks all thresholds
# 3. Returns list of alerts (or empty list if all normal)
```

Starter Code:

```
python
```

```python
def check_thresholds(reading):
    """
    Check sensor reading against thresholds

    Args:
        reading: dict with temp and humidity

    Returns:
        list of alert messages
    """
    alerts = []

    temp = reading['temperature']
    humidity = reading['humidity']

    # TODO: Check temperature thresholds
    # Your code here

    # TODO: Check humidity thresholds
    # Your code here

    # TODO: Check critical (both out of range)
    # Your code here

    return alerts

# Test data
test_readings = [
    {"device": "ESP32_001", "temperature": 25.5, "humidity": 60},  # Normal
    {"device": "ESP32_002", "temperature": 32.1, "humidity": 65},  # High temp
    {"device": "ESP32_003", "temperature": 16.5, "humidity": 25},  # Both low - CRITICAL
    {"device": "ESP32_004", "temperature": 28.0, "humidity": 75},  # High humidity
]

# Process each reading
for reading in test_readings:
    alerts = check_thresholds(reading)

    if alerts:
        print(f"Device {reading['device']}:")
        for alert in alerts:
            print(f"  ⚠️ {alert}")
    else:
        print(f"Device {reading['device']}: ✅ All normal")
    print()
```

Expected Output:

```
Device ESP32_001: ✅ All normal

Device ESP32_002:
  ⚠️ HIGH TEMPERATURE ALERT: 32.1°C

Device ESP32_003:
  ⚠️ CRITICAL ALERT: Temperature 16.5°C AND Humidity 25%
```

```
Device ESP32_004:
  ⚠ HIGH HUMIDITY ALERT: 75%
```

Extension Challenge:

- Add configurable thresholds (not hardcoded)
- Add alert priority levels (INFO, WARNING, CRITICAL)
- Count consecutive threshold violations
- Only alert if threshold exceeded for 3+ readings in a row

PRACTICE SCHEDULE (This Week)

Day 1 (Today): Exercises 1-2

- Exercise 1: Temperature Simulator (1 hour)
- Exercise 2: JSON Handler (1 hour)
- Total: 2 hours

Day 2: Exercise 3

- Multiple Sensor Parser (1.5 hours)
- Extensions (0.5 hours)
- Total: 2 hours

Day 3: Exercise 4

- File Reading (1 hour)
- CSV Analysis (1 hour)
- Total: 2 hours

Day 4: Exercise 5

- MQTT Simulator (1.5 hours)
- Extensions (0.5 hours)
- Total: 2 hours

Day 5: Exercise 6

- Energy Calculator (1 hour)
- Daily Cost Analysis (1 hour)
- Total: 2 hours

Day 6: Exercise 7

- Alert System (1.5 hours)
- Advanced Thresholds (0.5 hours)
- Total: 2 hours

Day 7: Mini Project

Combine everything into one script:

- Generate sensor data
- Store in JSON file
- Read and analyze
- Calculate energy costs
- Generate alerts
- Save report

Total: 12-14 hours of hands-on practice this week

HOW TO PRACTICE EFFECTIVELY

Setup Your Environment:

1. Create a folder for practice:

```
Documents/
  IoT-Learning/
    week1-python/
      exercise1.py
      exercise2.py
      sensor_log.csv
      README.md (your notes)
```

2. Use VS Code or any Python IDE

- Install Python extension
- Enable auto-complete
- Use terminal to run: `python exercise1.py`

3. Test immediately: Don't write 50 lines then test. Write 5 lines → test → next 5 lines.

The Learning Loop:

Level 1: Copy and understand

- Copy the starter code
- Run it
- Understand each line
- Add comments explaining what it does

Level 2: Modify

- Change variable names
- Adjust thresholds
- Add print statements to see values

Level 3: Extend

- Do the extension challenges
- Add your own features
- Break it intentionally and fix it

Level 4: Create from scratch

- Close the example
- Recreate it from memory
- This is when you've truly learned

COMMON MISTAKES & SOLUTIONS

Mistake 1: "I get errors and give up"

Solution: Errors are GOOD. They teach you.

- Read the error message carefully
- Google the exact error
- Use print() to debug
- Ask ChatGPT/Claude: "Why do I get this error: [paste error]"

Mistake 2: "I just copy-paste solutions"

Solution: That's not learning.

- Type every character yourself
- Change variable names
- Add your own twist

Mistake 3: "Exercises are boring, I want real projects"

Solution: These ARE real IoT code.

- Exercise 2 = How AWS Lambda processes IoT data
- Exercise 5 = How ESP32 publishes to MQTT
- Exercise 6 = How you prove ROI to clients
- Exercise 7 = How CloudWatch alarms work

Mistake 4: "I don't understand everything"

Solution: That's normal.

- You don't need to understand EVERYTHING
- Focus on: Can you use it?
- Understanding comes with repetition

AFTER THIS WEEK

You should be able to:

- ✅ Write Python scripts without help

- ✅ Work with dictionaries and lists (99% of IoT data)
- ✅ Read/write JSON files
- ✅ Process multiple sensor readings
- ✅ Create alert logic
- ✅ Calculate business metrics from sensor data

This prepares you for:

- Week 2: ESP32 coding (C++ but similar logic)
- Week 3: AWS Lambda functions (Python + these patterns)
- Week 4: Real IoT data pipelines

GETTING HELP

If stuck on an exercise:

Option 1: Google (Learn to search)

- "Python read CSV file"
- "Python calculate average from list"
- "Python check if value in range"

Option 2: ChatGPT/Claude (Get explanation)

```
Prompt: "I'm doing this exercise: [paste exercise]
I wrote this code: [paste your code]
I get this error: [paste error]
Can you explain what's wrong?"
```

Option 3: Python Documentation

- https://docs.python.org/3/tutorial/

Option 4: Stack Overflow

- Someone has asked your exact question before

SAVE YOUR WORK

After each exercise:

```bash
# In terminal/command prompt
cd week1-python
git add .
git commit -m "Completed Exercise 3 - Sensor data parser"
git push
```

Why:

- Track your progress

- Show on GitHub (even if private initially)
- Practice Git (part of your roadmap)

SUCCESS METRICS

After 7 days of practice, you should:

- [ ] Complete all 7 exercises
- [ ] Do at least 3 extension challenges
- [ ] Create one mini project combining everything
- [ ] Have 20-30 commits on GitHub
- [ ] Feel comfortable writing Python without constant Googling
- [ ] Understand 80% of Python IoT code you read online

If you can do this, you're ready for Week 2 (ESP32).

YOUR NEXT MESSAGE TO ME

After you complete Day 1 (Exercises 1-2), tell me:

1. Which exercise was easy?
2. Which one was hard?
3. What error did you struggle with most?
4. Did you do any extensions?
5. Screenshot of your working code output

Don't just read this. START CODING NOW.

Open VS Code. Create exercise1.py. Start typing.

You have 2 hours today. Go! 🚀